{% raw %}

ZAP! Language Reference Manual

The author of this software stands in solidarity with 🇺🇦 Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in 🇺🇦 Ukraine.

Complete Guide to the ZAP! Programming Language

Version: 1.0
Date: January 2026
Target Platforms: Atari 8-bit, 6502-based systems

Table of Contents

Getting Started

Your First ZAP Program

```
proc main()
end
```

This simple program defines a main procedure. Every ZAP program must have a `main` procedure as its entry point.

Compilation

```
python compiler.py program.zap -o program.s
```

This generates 6502 assembly code ready for linking with the Atari 8-bit development tools.

Basic Concepts

Program Structure

A ZAP program consists of:

- **Global variable declarations** - Data shared across procedures
- **Procedure definitions** - Blocks of code called by other procedures
- **Function definitions** - Procedures that return values
- **Module directives** - For multi-file compilation

Case Sensitivity

ZAP! is case-insensitive:

```
byte myVar
byte MYVAR      ; Same variable - error: duplicate!
byte x = myvar  ; Reference to myVar - valid
```

However, strings and character literals preserve case:

```
byte x = 'A'    ; Character 'A' (value 65)
byte msg[] = "Hello"  ; String preserved as-is
```

Comments

Single-line comments begin with `;` and extend to end of line:

```
proc main()
    ; This is a comment
    byte x = 5  ; Initialize x to 5
end
```

Semicolon Requirement

Statements do NOT require semicolons (they're optional and ignored).

Data Types

ZAP! supports three fundamental data types for 6502 systems. **All types are unsigned** — there are no signed integer types. Negative values are represented using two's complement bit patterns, just as the 6502 processor does natively.

byte - 8-bit Unsigned Integer

```
byte x              ; Uninitialized byte variable
byte y = 42         ; Byte with initializer
byte z = $FF        ; Hex notation
byte w = %10101010  ; Binary notation
byte n = -5         ; Negative literal: wraps to 251 ($FB) via two's
complement
```

Range: 0-255
Negative literals: Values -128 through -1 are valid and wrap modulo 256 (e.g., `-5` = 251).

word - 16-bit Unsigned Integer

```
word address        ; Uninitialized word variable
word counter = 1000 ; Word with initializer
word value = $1234  ; Hex notation (4 digits for 16-bit)
word neg = -200     ; Negative literal: wraps to 65336 ($FF38)
```

Range: 0-65535
Negative literals: Values -32768 through -129 that don't fit in a byte are typed as WORD.

long - 32-bit Unsigned Integer

```
long big_num        ; Uninitialized long variable
long population = 1000000
long mask = $FFFFFFFF
```

Range: 0-4294967295

Negative Numbers and Two's Complement

ZAP! has no signed integer types, but the 6502 processor represents negative values naturally using two's complement arithmetic. ZAP! supports negative integer literals as a shorthand for their two's complement bit patterns:

Literal	Type	Stored value
`-1`	byte	255 ($FF)
`-5`	byte	251 ($FB)

Literal	Type	Stored value
`-128`	byte	128 ($80)
`-200`	word	65336 ($FF38)

Negative literals in the range -128..−1 are classified as **byte**; those in -32768..−129 are **word**. When assigned to a wider type, the value is zero-extended (not sign-extended):

```
byte a = -5         ; a = 251 ($FB)  — byte two's complement
byte b = -1         ; b = 255 ($FF)
byte c = -128       ; c = 128 ($80)

; Runtime arithmetic wraps the same way:
byte d = 0 - 5      ; d = 251 ($FB)  — identical result
byte e = 200 + 56   ; e = 0 ($00)   — overflow wraps mod 256
```

> **Note**: Overflow and underflow at runtime always wraps modulo the type size (256 for byte, 65536 for word). No exception is raised.

Character Literals

Character literals are written with single quotes and are converted to their ASCII numeric values. A character literal always produces a **BYTE** value. Multi-character literals are not supported.

```
byte c = 'A'        ; Character literal (value 65)
byte newline = '\n' ; Newline character (value 10)
byte tab = '\t'     ; Tab character (value 9)
byte quote = '\''   ; Single quote (value 39)
byte backslash = '\\' ; Backslash (value 92)
byte null = '\0'    ; Null terminator (value 0)
byte hex_255 = '\xFF' ; Hex escape (value 255)
byte octal_65 = '\101' ; Octal escape (value 65, same as 'A')
byte binary_65 = '\b01000001' ; Binary escape (value 65, same as 'A')
```

Character Escapes (in character and string literals):

Standard Control Characters:

- `\n` - CRLF
- `\t` - Tab (4 columns)
- `\0` - Null byte (0)

Quote and Backslash:

- `\"` - Double quote
- `\'` - Single quote
- `\\` - Backslash

Numeric Escape Sequences:

- `\xHH` - Hexadecimal byte (1-2 hex digits: 0-9, a-f, A-F)
  - Example: `\xFF` (255), `\x41` (65/'A'), `\x0` (0)
- `\OOO` - Octal byte (1-3 octal digits: 0-7)
  - Example: `\377` (255), `\101` (65/'A'), `\0` (0)
- `\bBBBBBBBB` - Binary byte (1-8 binary digits: 0-1)
  - Example: `\b11111111` (255), `\b01000001` (65/'A'), `\b00000000` (0)

All numeric escapes produce a single **BYTE** value (0-255). Multibyte escape sequences are not supported — each escape encodes exactly one byte.

Non-ASCII characters: Raw non-ASCII characters (ordinal > 127) are **not** allowed directly in string or character literals. Use `\xHH` escapes for byte values 128–255. This includes platform-specific control characters such as the Atari EOL (`\x9B`).

Examples:

```
byte arr[] = "Hello\0World"       ; String with embedded null
byte data[] = "\xFF\x00\x42"      ; Binary data using hex escapes
byte atari_eol[] = "HELLO\x9B"    ; Atari end-of-line terminator ($9B)
byte mask = '\b11110000'           ; Mask value using binary
byte code = '\101'                 ; Letter 'A' using octal
```

The compiler emits string literals in readable ca65 assembly format, keeping printable ASCII as quoted strings and encoding non-printable or high bytes as individual `$XX` hex values:

```
; "HELLO\x9B" compiles to:
.byte "HELLO", $9B, $00
```

Type Modifiers

const - Constant Values

```
const byte VERSION = 1          ; Compile-time constant (byte)
const word MAX_SIZE = 1024      ; Compile-time constant (word)
const byte ^data_ptr = @buffer  ; Constant pointer
const byte arr[] = { 1, 2, 3 }  ; Constant array
```

The `const` modifier can be applied to any variable type. Constant values are evaluated at compile-time and cannot be modified at runtime. This includes:

- **Scalars** (byte, word)
- **Pointers** (byte ^, word ^)
- **Arrays** (array elements cannot be modified)

- **Structs** (struct instances cannot be modified, nor can their fields)

Attempting to modify a const variable results in a compile-time error:

```
const byte x = 10
x = 20              ; ERROR: Cannot assign to const

const byte arr[] = { 1, 2, 3 }
arr[0] = 5          ; ERROR: Cannot modify const array

struct Point
    byte x
    byte y
end

const Point p = { 5, 10 }
p.x = 20            ; ERROR: Cannot modify field of const struct
```

static - Static Local Variables

The `static` modifier creates local variables that retain their value between procedure calls. Static variables are initialized once at program startup (after global variables) and maintain their state across multiple invocations of the procedure.

Syntax:

```
proc counter()
    static byte count = 0       ; Initialized once at startup, retains
value
    count = count + 1
end

proc main()
    counter()  ; count becomes 1
    counter()  ; count becomes 2
    counter()  ; count becomes 3
end
```

port - Hardware Port Variables (#PORT, #RD, #WR) 🔌

Use `#PORT` to mark a variable as a hardware port-mapped variable. Port declarations are regular (global) variable declarations with an `@` address specifier and the `#PORT` modifier. Optionally attach `#RD` and/or `#WR` to indicate read/write capabilities.

Syntax:

```
byte POKEY_AUDF1 @$D200 #PORT        ; read/write port (default)
byte STATUS_PORT @$FFF0 #PORT #RD    ; read-only port
byte DATA_PORT @$D201 #PORT #WR      ; write-only port
```

Rules and semantic constraints (enforced by `sema.py`):

- `#PORT` requires an explicit address (`@ NUMBER`); e.g., `@$D200`.
- `#PORT` **cannot** be combined with `const` or `static`.
- `#PORT` cannot be used on arrays or pointer types.
- `#PORT` variables **cannot** have initializers (hardware ports cannot be initialized).
- `#RD` and `#WR` are only valid together with `#PORT`. If neither `#RD` nor `#WR` is specified, the port allows both read and write accesses by default.
- Attempting to `WRITE` to an `#RD`-only port or `READ` from a `#WR`-only port is flagged as a semantic error.

Use cases:

- Represent memory-mapped or I/O ports in platform-specific library modules.
- Document hardware access permissions with `#RD`/`#WR` so semantic checks can catch misuse.
- For devices with multiple registers, declare a `struct #PORT` and use field-level `#RD`/`#WR`. See **Port Structs** in the Structs chapter.

Examples:

```
byte POKEY_AUDF1 @$D200 #PORT       ; standard read/write
byte JOYSTICK @$D300 #PORT #RD      ; reading joystick status
byte SOUND_OUT @$D400 #PORT #WR     ; writing audio registers
```

Enums - Compile-time named constants 🧾

Enums provide a concise syntax for defining a set of named integer constants. They are compile-time only — no runtime storage or overhead.

Syntax (END-style):

```
enum [byte|word] Name
    ITEM1 [= expr]
    ITEM2
    ITEM3 = 5
    ITEM4
END
```

Key points:

- The base type is optional and defaults to `byte`. Use `word` when values may exceed the `byte` range.
- If a member has no explicit value, it takes the previous member's value + 1, or 0 for the first member.

- Values are range-checked against the chosen base type (`byte`: 0–255, `word`: 0–65535).
- Each member is a typed compile-time constant with the enum's base type (`byte` or `word`) — usable anywhere a `const` value is accepted.
- Duplicate member names within the same enum are reported as a compile-time error.
- Enums do not allocate runtime memory.

Access — qualified syntax only:

Enum members are accessible **exclusively** via the qualified `EnumName.Member` syntax. Unqualified access (using the member name alone) is not supported. This allows different enums to have members with the same name without any conflict.

```
enum Colors
    RED
    GREEN
    BLUE
END

enum Direction
    UP
    DOWN
    LEFT
    RIGHT
END

byte c = Colors.GREEN      ; c == 1
byte d = Direction.DOWN    ; d == 1 (no conflict with Colors.GREEN)
byte arr[Colors.BLUE + 1]  ; use enum value in an array size
```

Explicit values and auto-increment:

```
enum byte E
    A = 1
    B       ; B == 2
    C = 5
    D       ; D == 6
END

const byte v = E.D
```

Word-sized enum and large values:

```
enum word Big
    A = 300
    B
    C = 65535
END
```

```
const word w1 = Big.A
```

Common errors (semantic checks performed by `EnumAnalyzer` in `sema.py`):

- "Enum value N out of range for byte" — explicit or inferred value is outside 0..255 for `byte` enums.
- "Enum member 'NAME' duplicated in enum 'X'" — duplicate member names within the same enum.
- "Enum base type 'TYPE' is not supported" — only `byte` and `word` are supported base types.

Usage notes:

- Use enums for readable, self-documenting constants and to define sets of related identifiers or flags.
- Enum members can be used wherever `const` values are allowed: initializers, array dimensions, compile-time expressions, arithmetic expressions (+, -, *, /, %), logical expressions (&&, ||, !), bitwise expressions (&, |, ^, <<, >>) and comparison operations (<, >, ==, <=, >=).
- Only `.` is supported for qualification (colon `:` is not supported).

Tests:

- The test suite includes cases for: basic enums, enums with explicit values, word-sized enums, and failing cases for out-of-range values, duplicate members, and invalid base types.

Rules for static variables:

- Can **only be used on local variables** (inside procedures/functions)
- Cannot be combined with `const` modifier
- **Must have an initializer** - static variables require an initial value
- The initializer is evaluated once at program startup
- The variable retains its value between calls

Invalid usage:

```
static byte global_var = 5      ; ERROR: static only for local variables

proc test()
    static byte x              ; ERROR: static requires initializer
end

proc test2()
    static const byte y = 10   ; ERROR: cannot combine static and const
end
```

Common use cases:

1. **Call counters and sequences:**

```
proc get_next_id()
    static byte next_id = 1
    byte result = next_id
```

```
    next_id = next_id + 1
    return result
end
```

2. **State machines:**

```
proc update_game_state()
    static byte state = 0  ; 0=init, 1=running, 2=paused

    if state == 0
        initialize_game()
        state = 1
    elseif state == 1
        run_game()
    elseif state == 2
        handle_pause()
    end
end
```

3. **Resource tracking:**

```
func byte allocate_handle()
    static byte handle_count = 0

    if handle_count < 10
        byte h = handle_count
        handle_count = handle_count + 1
        return h
    end
    return 255  ; Error: no handles available
end
```

Declaration Modifiers (#KEEP, #NOEXPORT, #EXPORT, #PORT, #RD, #WR, #ASM) 🔖

ZAP! supports a set of trailing declaration modifiers that influence export behaviour, dead-code elimination, and port semantics. Modifiers are case-insensitive and are written after a declaration header (after the `)` of `proc`/`func`, or after a variable declarator list).

Examples:

```
proc atari_file_data_area() #KEEP #NOEXPORT
byte KEEPVAR #KEEP
const byte CVAL = 10 #EXPORT
byte POKEY_AUDF1 @$D200 #PORT #RD
byte STATUS_PORT @$FFF0 #PORT #RD
byte DATA_PORT @$D201 #PORT #WR
```

Modifiers:

- `#KEEP` — Prevents the symbol (procedure, function, global variable, or const) from being removed by dead-code elimination even if it is not referenced elsewhere.
- `#NOEXPORT` — When the file is declared as a `.module`, this prevents the symbol from being exported to files that include the module.
- `#EXPORT` — When the file is *not* a `.module`, this forces the symbol to be exported (useful for small libraries implemented in plain files).
- `#PORT` — Marks a variable as a hardware port-mapped variable. See the "Port Variables" section for details.
- `#RD` / `#WR` — Read/write qualifiers used together with `#PORT` to indicate whether the port is readable and/or writable. If neither `#RD` nor `#WR` is specified, both are allowed by default.
- `#ASM` — Marks a `proc` or `func` as a **pure assembly procedure/function**. The entire body between the declaration and `end` is treated as raw ca65 assembly text. All parameter name equates and local variable equates are still emitted before the body so that assembly code can reference ZAP-generated symbols. No automatic `RTS` is emitted. This is intended for interrupt service routines, hardware access routines, and other low-level code that must control its own exit sequence.

Pure Assembly Procedures and Functions (#ASM)

The `#asm` modifier turns the entire body into verbatim assembly code. The compiler still emits the standard infrastructure before the body:

- Parameter name equates (`_PROCNAME$PARAMNAME = actual_slot`) so the assembly can reference parameters by name
- Local variable equates for any declared locals
- Register → slot stores for register-passed arguments

After that infrastructure, the raw assembly body is emitted verbatim. No automatic `RTS` or return sequence is appended — the programmer is responsible.

Interrupt handlers (no parameters):

```
proc NMI_HANDLER() #keep #asm
    ; NMI handler — must return with RTI, not RTS
    rti
end

proc IRQ_HANDLER() #keep #asm
    ; Acknowledge interrupt, then return
```

```
    lda #$FF
    sta $D200   ; clear IRQ source
    rti
end
```

Parameterized proc — write a byte to an address:

```
proc my_poke(byte val) #asm
    ; _MY_POKE$VAL is an equate pointing to val's ZP slot
    lda _MY_POKE$VAL
    sta $4200
    rts
end
```

Pure-asm func — returns a value in register A:

```
func byte add_bytes(byte a, byte b) #asm
    ; _ADD_BYTES$A and _ADD_BYTES$B are equates for the ZP slots
    clc
    lda _ADD_BYTES$A
    adc _ADD_BYTES$B
    rts    ; result in A per ZAP calling convention
end
```

Rules and notes for `#asm` procedures and functions:

- `#asm` is valid on both `proc` and `func` declarations.
- Parameter name equates (`_PROCNAME$PARAM`) are emitted before the body so the assembly can reference ZAP parameters by name.
- The body may contain any ca65 assembly syntax: instructions, directives, labels, comments.
- **No automatic `RTS` is appended.** The programmer is responsible for the exit sequence (`rts`, `rti`, `jmp`, etc.).
- No local variable declarations or ZAP statements are allowed in the body.
- `#asm` may be combined with `#keep`, `#noexport`, and `#export`.
- The body is emitted verbatim into the output assembly surrounded by `; ASM_BLOCK_BEGIN` / `; ASM_BLOCK_END` markers.
- Optimization passes skip the body entirely.

Rules and notes (general modifiers):

- In a `.module` file, **all** top-level symbols are exported by default except those explicitly marked `#NOEXPORT`.
- In non-module files, **no** symbols are exported by default; use `#EXPORT` to explicitly export a symbol.
- `#KEEP` does **not** imply exporting; use `#EXPORT` if you want the symbol to be visible to includes.
- `#PORT`, `#RD`, and `#WR` are only valid on variable declarations (see "Port Variables" below). `#RD` and `#WR` are only valid when `#PORT` is present.

- Modifiers may be combined (e.g., `#KEEP #NOEXPORT`) and are parsed in any order.
- These modifiers apply to global declarations (variables, consts), and to top-level `proc`/`func` declarations. The parser emits these as `TOK_DECLMOD` tokens and the semantic checks enforce the constraints (see `parser.py` / `sema.py`).

Pointer Types

```
byte ^ptr           ; Pointer to byte
word ^addr          ; Pointer to word
struct Point ^p     ; Pointer to struct
```

See Pointers section for detailed information.

Variables

Variable Declaration

Basic syntax:

```
[const] type name [@ address] [= initializer]
```

Global Variables

```
byte global_x = 0                   ; Global variable
byte screen = 40000 @$9C00          ; Using hex address
word counter = 0
const byte VERSION = 1              ; Compile-time constant
```

Local Variables

```
proc main()
    byte local_var                  ; Local to main
    byte temp = 0                   ; Local with initialization
    const byte MAX_LOCAL = 100      ; Local constant

    ; Use variables here
end
```

Variable Scope

Global variables are visible from all procedures and functions.

Local variables are visible only within their procedure/function:

```
byte global_x = 10

proc test1()
    byte x = 20         ; Local x shadows global
    x = x + 1           ; Uses local x, not global
end

proc test2()
    x = x + 1           ; Uses global x
end
```

Fixed-Address Variables

Specify exact memory address for hardware registers or memory-mapped I/O:

```
byte SCREEN_DMA @$D400      ; ANTIC DMA control
word DISPLAY_LIST_PTR @$D402
byte PLAYER0_X @$D000       ; Atari GTIA register
```

Fixed-address variables are always preserved (never optimized away) regardless of usage.

Variable Initialization

Scalar initialization:

```
byte x = 42
word y = 1000
```

Array initialization (see Arrays):

```
byte arr[] = {1, 2, 3, 4, 5}
```

String initialization (see Strings):

```
byte text[] = "Hello"
```

Uninitialized Variable Detection

The ZAP! compiler performs **definite-assignment analysis** to detect when variables are read before initialization. This catches bugs at compile time:

```
proc safe()
    byte x = 0          ; Must initialize
    byte y = x + 10     ; OK: x is initialized
end

proc unsafe()
    byte x              ; Not initialized
    byte y = x + 10     ; ERROR: Use of uninitialized variable 'X'
end
```

The compiler tracks initialization through control flow:

```
proc example(byte flag)
    byte x

    if flag
        x = 10
    else
        x = 20
    end

    result = x          ; OK: x initialized in all paths
end
```

Behavior by type and context:

Type	Global	Local scalar	Local array	Local struct	Static local
byte / word / long	Zero (BSS)	**Compile error** if read before write	Zero on first call (BSS)	—	**Compile error** (initializer required)
pointer (byte^)	Zero/null (BSS)	**Compile error** if read before write	—	—	**Compile error**
array (byte[N], word[N])	Zero (BSS)	—	Zero on first call (BSS)	—	**Compile error**
struct	Zero (BSS)	—	—	Zero on first call (BSS)	**Compile error**

Additional rules:

- `const` variables, fixed-address (`@addr`) variables, and parameters are always considered initialized.
- FOR loop variables are initialized by the loop start expression.
- **Local arrays** are not checked by definite-assignment analysis — the array base identifier is always considered valid. Individual elements may read as zero on the first call (BSS), but retain their values on

subsequent calls.
- **Local structs** are always considered initialized by the compiler (a known limitation). Fields read as zero on the first call (BSS), but retain values on subsequent calls.
- **Static local** variables always require an explicit initializer — `static byte x` without `= value` is a compile error.
- Taking the address (`@`) of a variable does not require it to be initialized.

For complete details on the analysis rules, control flow handling, and limitations, see Safety Features.

Static Local Variables

Local variables maintain their values between procedure calls:

```
proc counter_increment()
    byte count          ; Persists between calls
    count = count + 1
end

proc main()
    counter_increment() ; count = 1
    counter_increment() ; count = 2
    counter_increment() ; count = 3
end
```

Note: Local variables are not automatically initialized on first call. Use an initialization flag if needed:

```
byte init_flag = 1

proc initialize_once()
    byte state

    if init_flag
        state = 0
        init_flag = 0
    end

    state = state + 1
end
```

Multiple Declarations

```
byte x, y, z           ; Three bytes
byte a = 1, b = 2      ; With initialization
byte ^p1, ^p2          ; Two byte pointers
```

Operators

Arithmetic Operators

Operator	Example	Result	Notes
`+`	`5 + 3`	`8`	Addition (8/16/32-bit)
`-`	`5 - 3`	`2`	Subtraction (8/16/32-bit)
`*`	`5 * 3`	`15`	Multiplication (8/16/32-bit)
`/`	`15 / 3`	`5`	Integer division
`%`	`17 % 5`	`2`	Modulo (remainder)

```
proc arithmetic_example()
    byte a = 10
    byte b = 3
    byte sum = a + b        ; 13
    byte diff = a - b       ; 7
    byte prod = a * b       ; 30
    byte quot = a / b       ; 3
    byte rem = a % b        ; 1
end
```

Expression Width and Promotion

ZAP evaluates arithmetic at 8-bit, 16-bit, or 32-bit width based on operand types and the assignment target.

**Rules (arithmetic operators +, -, , /, %):*

- If any operand is `long`, the expression is evaluated as **32-bit**.
- If any operand is `word` (or a pointer), the expression is evaluated as **16-bit** (unless there is a `long` operand).
- If all operands are `byte` and the assignment target is `byte`, the expression is evaluated as **8-bit**.
- If assignment target is wider than operands, operands are promoted to target width.
- If assignment target is narrower than operands, result is truncated.
- Pointer arithmetic is always **16-bit** because addresses are 16-bit.

Summary table:

Target	Operands	Evaluation Width	Result Behavior
`byte`	all `byte`	8-bit	Wraps 0-255
`word`	all `byte`	16-bit	Zero-extended
`long`	all `byte`	32-bit	Zero-extended
`byte`	any `word`/pointer	16-bit	Truncated to low byte

Target	Operands	Evaluation Width	Result Behavior
`word`	any `word`/pointer	16-bit	Full 16-bit result
`long`	any `word`/pointer	32-bit	Zero-extended
`any`	any `long`	32-bit	Truncated if target is narrower

This preserves carry/borrow for larger targets while keeping `byte`-only math compact and fast.

Comparison Operators

All comparison operators work on `byte`, `word`, and `long` operands. The result is always a `byte` (`0` for false, `1` for true).

Operator	Example	Meaning
`==`	`x == 5`	Equal
`!=`	`x != 5`	Not equal
`<`	`x < 5`	Less than
`>`	`x > 5`	Greater than
`<=`	`x <= 5`	Less than or equal
`>=`	`x >= 5`	Greater than or equal

```
proc comparison_example()
    byte x = 42

    if x == 42
        ; x equals 42
    end

    if x > 40 && x < 50
        ; x is between 40 and 50
    end
end
```

Logical Operators

Operator	Example	Meaning
`&&`	`a && b`	Logical AND (short-circuit)
`||`	`a || b`	Logical OR (short-circuit)
`!`	`!a`	Logical NOT

```
proc logical_example()
    byte x = 5
    byte y = 10

    ; AND operator
    if x > 0 && y > 0
        ; Both conditions true
    end

    ; OR operator
    if x == 5 || y == 5
        ; At least one condition true
    end

    ; NOT operator
    if !(x == 0)
        ; x is not zero
    end
end
```

Boolean Representation

ZAP! represents boolean results as **byte** values. The compiler emits `0` for false and `1` for true. When a boolean is used in a word context, it is zero-extended to 16 bits. This keeps logic operations compact while preserving correct behavior in mixed-width expressions.

Unary Operators

Operator	Example	Meaning	Result type
`-`	`-x`	Arithmetic negation	Same as operand (byte/word/long)
`!`	`!flag`	Logical NOT (0 → 1, non-zero → 0)	byte
`~`	`~mask`	Bitwise NOT	Same as operand (byte/word/long)
`@`	`@arr[i]`	Address-of	word pointer

The `-` and `~` operators preserve the operand type: negating a `word` gives a `word`, negating a `long` gives a `long`.

Negative literals vs. negation of a variable:

- `-5` as a **literal constant** is a BYTE with value 251 ($FB) — the two's complement bit pattern.
- `-x` where `x` is a **variable** computes `0 - x` at runtime, wrapping mod the variable's type size.

Both forms produce the same bit pattern; the distinction is only relevant when the compiler needs to determine the type of a standalone expression.

```
byte x = 5
byte neg_lit = -5       ; literal: byte 251 ($FB) — no runtime subtraction
```

```
byte neg_var = -x       ; runtime: 0 - x = 251 ($FB)
word wx = 1000
word wneg = -wx         ; word negation: 64536 ($FC18)
long lx = 100000L
long lneg = -lx         ; long negation wraps in 32-bit

byte flag = 1
byte notflag = !flag    ; logical NOT → 0
byte inv = ~flag        ; bitwise NOT → 254 ($FE)
```

> **Note**: Negative constant array indices are not allowed. `arr[-1]` is a compile-time error. Use pointer arithmetic (`ptr - 1`) for negative offsets.

Bitwise Operators

Bitwise operators perform bit-level manipulation on numeric values. They work on `byte`, `word`, and `long` operands. The result type matches the widest operand.

Operator	Example	Meaning
`&`	`a & b`	Bitwise AND
`|`	`a | b`	Bitwise OR
`^`	`a ^ b`	Bitwise XOR
`~`	`~a`	Bitwise NOT (unary)
`<<`	`a << b`	Bitwise Left Shift
`>>`	`a >> b`	Bitwise Right Shift

```
proc bitwise_example()
    byte mask = $0F
    byte value = $FF

    byte and_result = value & mask     ; $0F - AND operation
    byte or_result = value | mask      ; $FF - OR operation
    byte xor_result = value ^ mask     ; $F0 - XOR operation
    byte not_result = ~value           ; $00 - Bitwise NOT

    ; Common pattern: check if bit is set
    if value & $80
        ; High bit is set
    end

    ; Bitwise operations on long (32-bit)
    long flags = $FFFF0000
    long masked = flags & $00FF0000    ; $00FF0000
    long shifted = flags >> 8          ; $00FFFF00
    if flags & $01000000
        ; Bit 24 is set
```

```
    end
end
```

Address-Of Operator (@)

The `@` operator (when used in expressions) retrieves the address of a variable, array element, or struct field. This is distinct from the `@` address specifier used in declarations.

Operator	Example	Returns	Notes
`@`	`@var`	`word`	Address of variable
`@`	`@arr[i]`	`word`	Address of array element
`@`	`@struct.field`	`word`	Address of struct field

```
byte data = 42
word addr = @data              ; Get address of data

byte arr[] = { 1, 2, 3 }
word elem_addr = @arr[1]       ; Get address of arr[1] (value 2)

struct Point
    byte x
    byte y
end

Point p = { 10, 20 }
word x_addr = @p.x             ; Get address of p.x field
```

The address-of operator is particularly useful for passing variable addresses to assembly code or storing memory locations:

```
proc setup_pointers()
    byte buffer[256]
    word buf_addr = @buffer    ; Store buffer address

    ; Can pass to assembly routines
    asm
        LDA #<buf_addr
        STA BUFPTR
        LDA #>buf_addr
        STA BUFPTR+1
    end
end
```

Built-in Pseudofunctions

ZAP provides a few compiler built-ins that look like function calls but are handled directly by the compiler. They do not generate real function calls.

low(expr) / high(expr)

Extract the low or high byte of a value.

- Accepts any numeric expression (`byte`, `word`, `long`, or pointer).
- Returns a `byte`.
- For `byte` expressions, `high()` returns `0`.
- For `long` expressions: `low()` returns byte 0 (the least-significant byte), `high()` returns byte 1.
- Works in constant expressions (e.g., array sizes, addresses, const initializers).
- Works on struct fields, array elements, and dereferenced pointers.

Examples:

```
word addr = $1234
byte lo = low(addr)     ; $34
byte hi = high(addr)    ; $12

byte b = 7
byte h = high(b)        ; 0

long n = $12345678
byte lo = low(n)        ; $78  — byte 0 (lowest)
byte hi = high(n)       ; $56  — byte 1
```

loww(expr) / highw(expr)

Extract the low or high **word** (16-bit half) of a `long` value.

- Accepts a `long` expression only (compile error on `byte` or `word`).
- Returns a `word`: `loww()` returns bits 0–15 (bytes 0-1), `highw()` returns bits 16–31 (bytes 2-3).
- Works in constant expressions.
- Can be combined with `low()`/`high()` to extract any individual byte of a `long`:

Expression	Result for `$12345678`
`low(n)`	`$78` — byte 0
`high(n)`	`$56` — byte 1
`loww(n)`	`$5678` — bytes 0-1
`highw(n)`	`$1234` — bytes 2-3
`low(highw(n))`	`$34` — byte 2
`high(highw(n))`	`$12` — byte 3

```
long  varL = $12345678
word  varW
byte  varB

varW = highw(varL)      ; varW = $1234
varB = high(varW)       ; varB = $12  (byte 3 of varL)
varB = high(highw(varL)); varB = $12  (same, inline chain)
```

sizeof(StructName)

Returns the size of a struct type in bytes.

name - Argument must be a struct type name or a variable declared as that struct.

- Returns a `word`.
- Works in constant expressions.

Examples:

```
struct Point
    byte x
    byte y
end

const word PT_SIZE = sizeof(Point)
byte buffer[sizeof(Point)]

Point p
word sz = sizeof(p)
```

poke(addr, value)

Write a byte value to a memory address. Generates inline code — no function call overhead.

- `addr` is any expression evaluating to a word (the target memory address).
- `value` is any expression evaluating to a byte (the value to write).
- When both `addr` and `value` are constants, generates a direct `LDA #val / STA addr`.
- When `addr` is constant and `value` is variable, generates `STA addr` (absolute store).
- When `addr` is variable, uses indirect addressing via a ZP temporary.

```
POKE($D020, 0)              ; write 0 to address $D020
POKE(screen + offset, 255)  ; computed address
```

peek(addr)

Read a byte value from a memory address. Returns a `byte`.

- `addr` is any expression evaluating to a word (the source memory address).
- When `addr` is a constant, generates a direct `LDA addr` (absolute load).
- When `addr` is variable, uses indirect addressing via a ZP temporary.

```
byte val = PEEK($D01F)          ; read joystick register
byte ch  = PEEK(screen + pos)   ; read from computed address
if PEEK($D20F) & $04            ; check keyboard status
    ; key pressed
end
```

> **Note**: POKE/PEEK are compiler built-in keywords. They cannot be redefined as user procedures or functions. Their names are case-insensitive (POKE, Poke, poke all work).

Operator Precedence

From lowest to highest:

1. `||` (Logical OR)
2. `&&` (Logical AND)
3. `==`, `!=` (Equality)
4. `<`, `>`, `<=`, `>=` (Comparison)
5. `|` (Bitwise OR)
6. `^` (Bitwise XOR)
7. `&` (Bitwise AND)
8. `+`, `-` (Addition, Subtraction)
9. `+`, `-` (Addition, Subtraction)
10. `<<`, `>>` (Bitwise Shift)
11. `*`, `/`, `%` (Multiplication, Division, Modulo)
12. `-`, `!`, `~`, `@` (Unary)
13. Primary (literals, variables, parentheses)

```
proc precedence_example()
    byte result

    ; Standard precedence
    result = 2 + 3 * 4      ; 14 (multiply first)
    result = (2 + 3) * 4    ; 20 (parentheses first)

    ; Logical precedence
    byte a = 1, b = 0, c = 1
    if a && b || c          ; (a && b) || c = true
    end

    ; Bitwise precedence
```

```
    result = 5 & 3 | 1      ; ((5 & 3) | 1) = (1 | 1) = 1
end
```

Compound Assignment Operators

ZAP supports C-style compound assignment as syntax sugar. `lhs op= expr` is exactly equivalent to `lhs = lhs op expr` — the desugaring happens at parse time so all existing type-checking rules and code-generation optimisations apply unchanged.

Operator	Equivalent to	Types
`x += expr`	`x = x + expr`	byte, word, long, pointer (`+=` only)
`x -= expr`	`x = x - expr`	byte, word, long, pointer (`-=` only)
`x *= expr`	`x = x * expr`	byte, word, long
`x /= expr`	`x = x / expr`	byte, word, long
`x %= expr`	`x = x % expr`	byte, word, long
`x &= expr`	`x = x & expr`	byte, word, long
`x |= expr`	`x = x | expr`	byte, word, long
`x ^= expr`	`x = x ^ expr`	byte, word, long
`x <<= expr`	`x = x << expr`	byte, word, long
`x >>= expr`	`x = x >> expr`	byte, word, long

All lvalue forms are supported on the left-hand side: plain variables, array elements, struct fields, and pointer dereferences.

```
proc compound_assign_example()
    byte b = 10
    word w = 1000
    long l = 65536
    byte arr[4] = {1, 2, 3, 4}
    byte ^ptr

    b += 5          ; b = 15
    b *= 2          ; b = 30
    b >>= 1         ; b = 15

    w += 256        ; w = 1256
    w &= $00FF      ; w = 232 ($E8)

    l <<= 1         ; l = 131072
    l -= 1          ; l = 131071

    arr[1] += 10    ; arr[1] = 12
```

```
    ptr = @arr
    ptr += 2        ; advance pointer by 2 elements
end
```

Control Flow

if-else Statement

```
proc if_example()
    byte x = 5

    ; Simple if
    if x == 5
        ; This executes
    end

    ; if-else
    if x > 10
        ; x is greater than 10
    else
        ; x is 10 or less
    end

    ; Nested if
    if x > 0
        if x < 100
            ; x is between 0 and 100
        end
    end

    ; if-elseif-else chain
    if x == 1
        ; handle case 1
    elseif x == 2
        ; handle case 2
    elseif x == 3
        ; handle case 3
    else
        ; handle all other cases
    end
end
```

The `elseif` keyword chains multiple conditions without nesting. Any number of `elseif` branches may appear between `if` and the optional `else`. Each condition is tested in order; the first true branch executes and the rest are skipped.

Switch Statement

The `switch` statement selects a code block to execute based on the value of an expression. It supports `case` labels for specific values and an optional `default` label.

Syntax:

```
switch expression
    case value1
        statement1
        break
    case value2
        statement2
        break
    default
        statement3
end
```

Key Features:

1. **Expression Type**: The switch expression can be `byte`, `word`, or `long` (or compatible types).
2. **Case Labels**: Must be constant expressions known at compile time.
3. **Fall-through**: ZAP! `switch` statements have **C-like fall-through behavior**. If a `case` block does not end with `break`, execution continues into the next `case` block.
4. **Default**: The `default` block executes if no case matches. It is optional.
5. **Break**: The `break` statement exits the `switch` structure.
6. **Continue**: The `continue` statement inside a `switch` is **not** a switch operation — it targets the nearest *enclosing loop* and jumps to its next iteration. This lets you skip the rest of the current loop iteration from inside a switch case.
7. **One-time dispatch**: The switch expression is evaluated once before any case body runs. Modifying the switch variable inside a case body does not cause a different case to be selected.

Example with Break (No Fall-through):

```
byte x = 1

switch x
    case 1
        x = 10
        break       ; Exit switch
    case 2
        x = 20
        break       ; Exit switch
    default
        x = 0
end
; Result: x is 10
```

Example with Fall-through:

```
byte x = 1
byte y = 0

switch x
    case 1
        y = 1       ; No break, falls through to case 2
    case 2
        y = 2       ; Executes if x is 1 OR 2
        break
    default
        y = 0
end
; Result: If x is 1, y becomes 2.
```

Example without `default` (no match → continues normally):

```
byte x = 7
byte hit = 0

switch x
    case 1
        hit = 1
        break
    case 2
        hit = 2
        break
end
; x is 7, no case matches, no default → hit stays 0
; Execution continues here normally
```

When no case matches and there is no `default`, the entire switch body is skipped and execution resumes at the statement after `end`.

Variable mutation inside a case body:

The switch expression is evaluated **once** before any case body runs. All case comparisons (the dispatch table) are resolved at that point. Modifying the switch variable inside a case body has no effect on which case was selected:

```
byte var = 1

switch var
    case 1
        var = 2         ; changes var, but dispatch already matched case 1
        break           ; exits switch — case 2 is NOT executed
    case 2
        ; This does NOT run, even though var is now 2.
        ; The dispatch already chose case 1 before this body ran.
```

```
end
; var is 2 here (modification is retained after the switch)
```

Fall-through is sequential code flow, **not** a re-evaluation of the dispatch. If `case 1` falls through (no `break`) into `case 2`, `case 2`'s body runs because the code is physically next — not because the variable's new value matches `case 2`:

```
byte var = 1
byte cnt = 0

switch var
    case 1
        var = 2          ; modifies var
        cnt = cnt + 1    ; cnt = 1
    case 2               ; reached by FALL-THROUGH (no break above)
        cnt = cnt + 1    ; cnt = 2 — not because var == 2
        break
end
; cnt is 2
```

`continue` inside `switch`: When a `switch` is nested inside a loop, `continue` inside a `case` body skips the rest of the **current loop iteration** — not a switch operation. The `switch` itself has no concept of "next case"; only `break` exits it.

```
proc continue_in_switch()
    byte i = 0
    byte cnt = 0

    while i < 5
        i = i + 1
        switch i
        case 3
            continue     ; skip cnt++ for i==3; jumps to while condition
check
        end
        cnt = cnt + 1    ; runs for i=1,2,4,5 — skipped when i==3
    end
    ; cnt == 4
end
```

while Loop

```
proc while_example()
    byte count = 0

    while count < 10
```

```
        count = count + 1
    end

    ; Loop with break
    byte x = 0
    while 1         ; Infinite loop
        x = x + 1
        if x == 100
            break   ; Exit loop
        end
    end
end
```

repeat-until Loop

`repeat` always runs the body at least once, then evaluates the condition at the end. The loop stops when the condition is true.

```
proc repeat_example()
    byte count = 0

    repeat
        count = count + 1
    until count == 5
end
```

`break` exits the loop immediately. `continue` skips the rest of the body and **jumps directly to the `until`-condition** (not back to the top of the body):

```
proc repeat_break_continue()
    byte i = 0

    ; break: stop early before condition is reached
    repeat
        i = i + 1
        if i == 3
            break           ; exits loop; i stays 3, not 10
        end
    until i == 10

    ; continue: jump to the until-condition, skipping code below it
    byte j = 0
    byte cnt = 0
    repeat
        j = j + 1
        if j == 2
            continue        ; skips cnt++ and re-evaluates until j==4
        end
        cnt = cnt + 1       ; runs on j=1,3,4 but NOT j=2
```

```
    until j == 4
    ; result: cnt == 3 (not 4), j == 4
end
```

Note: `break` and `continue` inside nested loops always target the **innermost** enclosing loop. A `break` inside a `while` nested in a `repeat` exits the `while` only; the `repeat` keeps running.

for Loop

```
proc for_example()
    byte i

    ; Basic for loop
    for i = 0 to 9
        ; Execute 9 times (0-8)
    end

    ; For loop with step
    for i = 0 to 100 step 10
        ; i = 0, 10, 20, ..., 90
    end

    ; Descending (use while for count-down)
    i = 10
    while i > 0
        ; i = 10, 9, 8, ..., 1
        i = i - 1
    end

    ; With break
    for i = 0 to 255
        if i == 128
            break
        end
    end
end
```

Semantics: The `to` bound is exclusive (C-like). The loop runs while `i < end`. Only positive step values are supported; use `while` for counting down. To include a specific last value, set `to` one step past it (e.g., `for i = 0 to 10` includes 9 with step 1; `for i = 0 to 110 step 10` includes 100).

Data types: The loop variable may be `byte`, `word`, `long`, or a pointer (`byte^`, `word^`). The `to` bound and `step` are widened to match the loop variable type.

LONG bounds: The loop variable, `to`-bound, and `step` may all be `long`. The compiler allocates 4-byte temporaries for the bounds automatically.

```
proc long_for_example()
    long i
```

```
    long start_val = 65536       ; Above word range
    long end_val   = 65540

    for i = start_val to end_val step 1
        ; Iterates 4 times: i = 65536, 65537, 65538, 65539
    end
end
```

Expression bounds and variable step: Both the `to` bound and the `step` may be runtime expressions or variables, not just constants.

```
proc expr_bounds_example()
    byte i
    byte a = 2
    byte b = 5
    word w
    word ws = 5

    ; Bounds from compound expressions
    for i = a + 1 to b * 2     ; start=3, end=10, runs 7 times
        ; i = 3, 4, 5, 6, 7, 8, 9
    end

    ; Step from a variable (general path)
    for w = 0 to 20 step ws    ; ws=5, runs 4 times
        ; w = 0, 5, 10, 15
    end
end
```

Pointer as loop variable: A pointer variable can be used as the loop variable to iterate over memory addresses.

```
proc pointer_loop_example()
    byte arr[4]
    byte^ ptr

    ; ptr iterates over arr[0]..arr[3] addresses
    for ptr = @arr[0] to @arr[0] + 4
        ptr^ = 1           ; Write 1 to each element
    end
    ; arr is now [1, 1, 1, 1]
end
```

Dereferenced pointer as end bound: A dereferenced pointer (`ptr^`) may appear in the `to` expression.

```
proc deref_end_example()
    byte target = 6
```

```
    byte^ ptr
    byte i
    byte cnt

    ptr = @target           ; ptr points to target (= 6)
    cnt = 0
    for i = 0 to ptr^       ; end = *ptr = 6, runs 6 times
        cnt = cnt + 1        ; i = 0, 1, 2, 3, 4, 5
    end
    ; cnt == 6
end
```

switch Statement

ZAP supports C-style `switch` with `case`, `default`, fallthrough, and `break`.

```
switch expr
    case value1
        ; statements
        break
    case value2
    case value3
        ; stacked labels
        break
    default
        ; optional default
        break
end
```

Rules:

- `case` labels must be compile-time constants.
- `default` is optional but can appear only once.
- Duplicate `case` values are an error.
- Fallthrough is allowed: if a `case` body does not end with `break`, execution continues into the next case.
- `break` exits the nearest `switch` or loop.

Example:

```
switch ch
    case 13
        putchar(10)   ; CRLF
        putchar(13)
        break
    case 8
        putchar(8)    ; Backspace
        putchar(32)   ; Space to clear character
        putchar(8)    ; Move back again
```

```
        break
    case 'a'
    case 'b'
    case 'c'
        PLAYF4 = COLOR_RED1 + (ch - 'a') * 2
        break
    default
        putchar(ch)
        break
end
```

break Statement

Exits the nearest enclosing loop (`while`, `for`, `repeat-until`) or `switch`. When loops are nested, `break` exits only the innermost one.

```
proc break_example()
    ; In while loop
    byte x = 0
    while x < 1000
        x = x + 1
        if x == 500
            break
        end
    end

    ; In for loop
    byte i
    for i = 0 to 255
        if i == 50
            break
        end
    end

    ; In repeat-until loop
    byte k = 0
    repeat
        k = k + 1
        if k == 3
            break       ; exits loop; k stays 3, not 10
        end
    until k == 10
end
```

continue Statement

Skips the remainder of the current loop body and moves to the **next iteration**:

- In `while` and `for`: jumps back to the loop condition check.
- In `repeat-until`: jumps directly to the `until`-condition (not back to the body start).

When loops are nested, `continue` affects only the **innermost** enclosing loop.

```
proc continue_example()
    ; In while loop: skip even numbers
    byte i = 0
    byte sum = 0
    while i < 10
        i = i + 1
        if i & 1 == 0
            continue    ; skip even i — jumps back to while condition
        end
        sum = sum + i   ; only odd i values accumulated
    end

    ; In for loop: continue jumps to the for condition check,
    ; which SKIPS the step increment for that iteration.
    ; Avoid continue in for loops unless the step is handled manually
before continue.
    for i = 1 to 10
        if i % 3 == 0
            continue    ; jumps to condition check — step increment is
skipped!
        end
        sum = sum + i
    end

    ; In repeat-until: continue jumps to until-condition
    byte j = 0
    byte cnt = 0
    repeat
        j = j + 1
        if j == 2
            continue    ; jumps to [until j == 5], skipping cnt++
        end
        cnt = cnt + 1
    until j == 5
    ; cnt == 4 (j=1,3,4,5 each hit cnt++; j=2 was skipped)
end
```

Zero/Non-Zero Evaluation

In conditional statements (`if`, `while`, `repeat-until`), the condition expression evaluates as:

- **Zero** - False
- **Non-zero** - True

This rule applies to **all integer types**: `byte`, `word`, and `long`.

```
proc zero_evaluation()
    byte x = 0
```

```
    byte y = 1

    if x              ; False (x is 0)
    end

    if y              ; True (y is non-zero)
    end

    if 0              ; False
    else
        ; This executes
    end

    if 1              ; True
        ; This executes
    end
end
```

LONG (32-bit) Truthiness

For `long` variables, **all four bytes are tested together**: the compiler OR-s all four bytes of the value and checks whether the result is non-zero. This means a `long` value like `65536` (`$00010000`) correctly evaluates as **True**, even though its low byte is `0`.

```
proc long_truthiness()
    long counter = 65536    ; Value in bytes 2-3 only, low bytes are 0

    if counter              ; True — upper bytes are non-zero
        ; This executes
    end

    while counter           ; Loops while any of the 4 bytes is non-zero
        counter = counter - 1
    end

    long flags = $01000000  ; Only byte 3 is set
    repeat
        flags = flags - $01000000
    until flags             ; Stops when all 4 bytes become zero
end
```

> **Note:** Without the 4-byte OR test, a `long` value of `65536` would appear `False` because its low byte is `0`. ZAP! handles this correctly for all control-flow statements.

Procedures & Functions

Procedures

Procedures are blocks of code that perform actions but don't return values.

```
proc procedure_name()
    ; Procedure body
end

proc procedure_with_params(byte x, word y)
    ; Use parameters
end

proc main()
    procedure_name()            ; Call without params
    procedure_with_params(5, 1000)  ; Call with params
end
```

Functions

Functions return values to their caller.

```
func byte add_one(byte x)
    return x + 1
end

func word multiply(byte a, byte b)
    return a * b
end

proc main()
    byte result = add_one(10)   ; result = 11
    word prod = multiply(5, 6)  ; prod = 30
end
```

Parameters

```
proc three_params(byte x, word y, byte c)
    ; Use x, y, c
end

; Parameters are procedure locals
proc param_example(byte value)
    byte value      ; ERROR: duplicate declaration
end

; But you can declare other locals
proc param_with_locals(byte value)
    byte temp = 0   ; OK: different name
end
```

Argument Width Rules

Arguments must match or be narrower than the declared parameter type. Passing a wider type is a compile-time error to prevent silent data truncation. Use explicit narrowing functions to convert:

```
proc draw(byte x, byte y)
end

word wx = $1234
long lx = $12345678

; Valid — same width or narrower
draw(10, 20)          ; constants fit in BYTE — OK
draw(LOW(wx), HIGH(wx))  ; explicit narrowing — OK

; Invalid — wider than parameter
draw(wx, 20)          ; ERROR: cannot pass WORD to BYTE
draw(lx, 20)          ; ERROR: cannot pass LONG to BYTE
```

Narrowing functions:

From	To	Use
WORD	BYTE	`LOW(expr)` or `HIGH(expr)`
LONG	WORD	`LOWW(expr)` or `HIGHW(expr)`
LONG	BYTE	`LOW(LOWW(expr))`, `HIGH(LOWW(expr))`, etc.

Exception: Integer constants that fit in the parameter's range are accepted without narrowing (e.g., `255` for BYTE, `65535` for WORD).

Default Parameters

Parameters can have default values. Parameters with defaults must follow all required parameters:

```
proc draw(byte x, byte y, byte color = 1)
    ; color defaults to 1 if omitted
end

func byte clamp(byte val, byte lo = 0, byte hi = 255)
    if val < lo
        return lo
    end
    if val > hi
        return hi
    end
    return val
end
```

```
proc main()
    draw(10, 20)        ; color = 1 (default)
    draw(10, 20, 3)     ; color = 3

    byte r = clamp(50)          ; lo=0, hi=255
    byte s = clamp(50, 10)      ; lo=10, hi=255
    byte t = clamp(50, 10, 100) ; lo=10, hi=100
end
```

Skipping Arguments

When calling a procedure or function with default parameters, you can skip individual arguments by leaving them empty between commas. The skipped parameter uses its default value:

```
proc setup(byte mode = 0, byte speed = 5, byte flags = $FF)
    ; ...
end

proc main()
    setup(1,,3)     ; mode=1, speed=5 (default), flags=3
    setup(,2)       ; mode=0 (default), speed=2, flags=$FF (default)
    setup(,,7)      ; mode=0 (default), speed=5 (default), flags=7
    setup(1,,,)     ; ERROR: more commas than parameters
end
```

The same syntax works for function calls in expressions:

```
func byte calc(byte a = 10, byte b = 20, byte c = 30)
    return a + b + c
end

proc main()
    byte r = calc(1,,3)  ; a=1, b=20 (default), c=3 → 24
end
```

Rules:

- A skipped argument must have a corresponding default value in the declaration.
- Trailing arguments with defaults can simply be omitted (no trailing commas needed).

Local Variables

```
proc with_locals()
    byte x          ; Local variable
    byte y = 10     ; Local with initializer
    word counter = 0
```

```
    ; Use locals
    x = y + 1
    counter = counter + 1
end
```

Return Statement

In procedures (returns to caller, no value):

```
proc early_exit(byte x)
    if x == 0
        return      ; Exit early
    end

    ; More code
end
```

In functions (required, must have value):

```
func byte get_value()
    byte x = 42
    return x
end

func word calculate(byte a, byte b)
    return a + b
end
```

Return Type Validation

The compiler checks that the return expression is compatible with the declared return type:

- **Scalar widening** (BYTE → WORD, BYTE → LONG, WORD → LONG): allowed — zero-extended automatically.
- **Scalar narrowing** (WORD → BYTE, LONG → BYTE, LONG → WORD): allowed — truncated to lower bytes.
- **Pointer ↔ WORD**: pointers are 2-byte values and are fully compatible with WORD. A `func word` may return a pointer, and a `func byte^` may return a word value.
- **Struct return**: the returned expression must be the exact same struct type, or a struct literal `{...}`. Returning a different struct type is an error.
- **Struct ↔ scalar mismatch**: returning a struct where a scalar is expected (or vice versa) is an error.
- **Missing expression**: `return` without an expression in a function is an error.
- **Constant range check**: if the return expression is a compile-time constant, it must fit in the declared type (e.g., returning 256 from a `func byte` is an error).

```
; These are compile-time errors:
func byte bad_struct()
    Point p
    return p            ; error: expected BYTE, got struct 'POINT'
end

func Point bad_scalar()
    return 42           ; error: expected struct 'POINT', got BYTE
end

func byte bad_range()
    return $0100        ; error: 256 does not fit in BYTE (0-255)
end

func byte bad_empty()
    return              ; error: RETURN in function must have an expression
end
```

Parameter Passing

Parameters are passed by value (copies created):

```
proc modify(byte x)
    x = 0           ; Modifies local copy only
end

#### Register Calling Convention (PROC/FUNC)

ZAP uses a lightweight register convention for the first parameters, with
the rest passed via parameter globals.

- **1 BYTE parameter**: passed in `A`
- **1 WORD parameter**: passed in `A`/`X` (low in `A`, high in `X`)
- **2 BYTE parameters**: passed in `A`/`X` (param0 in `A`, param1 in `X`)
- **3 BYTE parameters**: passed in `A`/`X`/`Y` (param0 in `A`, param1 in
`X`, param2 in `Y`)
- **WORD + BYTE (first two parameters)**: WORD in `A`/`X`, BYTE in `Y`
- **More parameters**: first parameters follow the rules above; remaining
parameters are passed via `_PROC$PARAM` / `_FUNC$PARAM` locals as before.

At function/procedure entry, register-passed values are stored into the
usual parameter locals so existing code sees the same variables.

#### Return Values

- **BYTE return**: `A`
- **WORD return**: `A`/`X` (low in `A`, high in `X`)

When used in a wider context, BYTE results are zero-extended to WORD (reg.
X) by the caller as needed.
```

```
proc main()
    byte value = 42
    modify(value)
    ; value still 42 - not affected by modify
end

#### Deep Copy Semantics

ZAP passes parameters **by value**. This means the callee receives its
**own copy** of the argument.

- **Scalars (byte/word)**: copied by value.
- **Structs**: the entire struct is copied (byte-for-byte). Changes inside
the callee do not affect the caller.
- **Pointers**: the pointer value (address) is copied, **not** the data it
points to. Dereferencing the pointer can modify the original data.
- **Arrays/strings**: there is no automatic deep copy. To work on array
data, pass a pointer (and size if needed).

Example: struct copy vs pointer access

```zap
struct Point
    byte x
    byte y
end

proc move_local(Point p)
    p.x = p.x + 1      ; Modifies local copy only
end

proc move_in_place(Point ^p)
    p^.x = p^.x + 1    ; Modifies caller's struct via pointer
end

proc main()
    Point a = { 10, 20 }
    move_local(a)
    ; a still {10, 20}
    move_in_place(@a)
    ; a now {11, 20}
end
```

Struct Literals as Call Arguments

You can pass a struct literal directly to a `proc` or `func` parameter that expects a struct value. The literal is copied into the parameter storage before the call.

```
struct Pair
    byte x
```
```

```
    byte y
end

struct WPair
    word wx
    word wy
end

struct Nested
    byte tag
    Pair p
end

func byte add_pair(Pair p)
    return p.x + p.y
end

func word add_wpair(WPair p)
    return p.wx + p.wy
end

func byte nested_sum(Nested n)
    return n.tag + n.p.x + n.p.y
end

proc main()
    byte a = add_pair({10, 20})         ; byte struct literal → a = 30
    word b = add_wpair({300, 400})      ; word struct literal → b = 700
    byte c = nested_sum({1, {10, 20}})  ; nested struct literal → c = 31

    ; struct literal can appear inline in any expression
    if add_pair({3, 4}) == 7
        ; ...
    end
end
```

Notes:

- The literal values must correspond to the struct fields in declaration order.
- Each value is assigned to the matching field: `byte` fields take byte values, `word` fields take word values.
- Nested struct fields use a nested literal `{outer_field, {inner_field, ...}}`.
- For in-place updates, pass a pointer (`Pair ^`) instead.

Recursive Calls

Procedures can call themselves:

```
proc countdown(byte n)
    putc(n + 48)    ; Print digit
```

```
    if n > 0
        countdown(n - 1)
    end
end

proc main()
    countdown(5)    ; Prints: 5 4 3 2 1 0
end
```

Note: parameters and local variables are not stored on the stack, so recursive calls overwrite them; true recursion is not supported.

Arrays & Strings

Array Declaration

```
byte arr1[10]                   ; Array of 10 bytes, uninitialized
byte arr2[10]                   ; Uninitialized (BSS zeroed)
word arr3[5]                    ; Array of 5 words
const byte arr4[] = {1, 2, 3}   ; Constant array (cannot modify)
```

Array Initialization

```
; With initializer list
byte values[] = {1, 2, 3, 4, 5}

; With specified size (fills with values)
byte data[8] = {255, 0, 127}    ; Rest are 0

; Word arrays
word addresses[] = {$2000, $3000, $4000}

; Constant arrays
const byte default_levels[] = {1, 2, 3, 4}
```

Array Subscripting

```
proc array_example()
    byte arr[5] = {10, 20, 30, 40, 50}
    byte i
    byte value

    ; Read from array
    value = arr[0]      ; value = 10
    value = arr[2]      ; value = 30
```

```
    ; Write to array (not allowed for const arrays)
    arr[1] = 99

    ; Dynamic subscripts
    for i = 0 to 4
        arr[i] = i * 10
    end
end
```

Index type: Any integer type (`byte`, `word`, `long`) may be used as an array index. Only the low bytes needed for address calculation are used — no runtime bounds check is performed (same as `word`-to-`byte` truncation). For `long` indices, only the low 16 bits are used; values ≥ 65536 wrap silently. This matches how WORD indices work with BYTE-sized arrays.

Array Address-Of Operator

Get the address of an array or array element using the `@` operator:

```
proc array_addressing()
    byte arr[] = {10, 20, 30, 40, 50}

    word arr_addr = @arr        ; Address of first element
    word elem_addr = @arr[2]    ; Address of arr[2]

    ; Useful for passing to assembly routines
    asm
        LDA #<arr_addr
        STA PTR
        LDA #>arr_addr
        STA PTR+1
    end
end
```

Strings

Strings are byte arrays with NUL termination:

```
byte message[] = "Hello"    ; Stored as {72,101,108,108,111,0}
```

```
proc string_example()
    byte greeting[] = "Hi"

    ; Access characters
    byte first = greeting[0]    ; 'H' = 72
    byte second = greeting[1]   ; 'i' = 105
```

```
    byte nul = greeting[2]      ; 0 (terminator)
end
```

Array Address Specification

```
byte screen_buffer[256] @$8000     ; Fixed address
word sprite_data[32] @$6000        ; Fixed address for word array
```

Multidimensional Arrays

Multidimensional Arrays

ZAP! supports multidimensional arrays of any depth. They are stored in row-major order (C-style).

Declaration

```
byte grid[3][4]              ; 3 rows, 4 columns (12 bytes)
word matrix[2][3]            ; 2x3 array of words
struct Point map[5][10]      ; 2D array of structs
```

Accessing Elements

```
proc access_grid()
    byte val
    val = grid[1][2]         ; Read element at row 1, col 2
    grid[0][0] = 42          ; Write element
end
```

Initialization

Nested initializer lists are supported:

```
byte weights[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
}
```

Partial Subscripting

Providing fewer indices than dimensions results in a pointer to the sub-array (e.g., a row):

```
byte ^row_ptr = grid[1]      ; Points to start of row 1
```

Structs

Structs are composite data types that group multiple fields of different types together. They allow you to create structured data, similar to records or objects in other languages.

Struct Definition

```
struct Point
    byte x
    byte y
end

struct Player
    byte x
    byte y
    byte health
    word score
end
```

Constraints

- **Maximum size**: A struct may not exceed **255 bytes** total. This limit exists because field offsets are loaded as 8-bit immediate values (`LDY #offset`). The compiler rejects struct definitions that exceed this limit:

  ```
  error: Struct 'BIGSTRUCT' is 256 bytes — maximum struct size is 255
  bytes
  ```

- Fields may be `byte`, `word`, `long`, pointer types (`byte^`, `word^`, `long^`, or a pointer to any struct type), nested struct types, or arrays of those types.

Pointer Fields

A struct field can hold the address of another variable or struct instance. The field occupies 2 bytes (a 16-bit address). All pointer operations — assignment of an address, dereferencing, and field access through a pointer — work exactly as they do for standalone pointer variables.

```
struct Target
    byte  val
    word  wval
end
```

```
struct Holder
    byte^   bptr    ; pointer to a byte variable
    word^   wptr    ; pointer to a word variable
    Target^ sptr    ; pointer to a Target struct
end

byte   bdata = 10
word   wdata = 1000
Target tdata = { 55, 300 }
Holder h

proc use_pointer_fields()
    ; Assign addresses to pointer fields
    h.bptr = @bdata
    h.wptr = @wdata
    h.sptr = @tdata

    ; Write through a pointer field
    h.bptr^ = 20        ; bdata is now 20
    h.wptr^ = 2000      ; wdata is now 2000

    ; Read through a pointer field
    byte  b = h.bptr^   ; b == 20
    word  w = h.wptr^   ; w == 2000

    ; Access a field of the pointed-to struct
    h.sptr^.val = 77    ; tdata.val is now 77
    byte v = h.sptr^.val
end
```

A pointer field to a struct type enables access to the target struct's fields using the `ptr^.field` syntax. Note that the compiler does not perform lifetime or aliasing checks; the programmer is responsible for ensuring that the pointed-to variable remains in scope.

Struct Initialization

```
; Initialize with values
Point p1 = { 10, 20 }

; Initialize with complex expressions
Point p2 = { 100 + 50, 200 - 50 }

; Nested: Structs containing other structs
struct Rectangle
    Point top_left
    Point bottom_right
end

Rectangle r = { { 0, 0 }, { 100, 100 } }
```

Field Access

```
proc struct_fields()
    Point p = { 15, 30 }

    ; Read fields
    byte x_value = p.x
    byte y_value = p.y

    ; Write to fields
    p.x = 20
    p.y = 40

    ; Nested field access
    Rectangle r = { { 0, 0 }, { 100, 100 } }
    byte top_left_x = r.top_left.x
    r.bottom_right.y = 50
end
```

Array Fields

A struct field can itself be an array. When initializing such a struct, the array field requires its own nested braces inside the struct initializer — the outer braces belong to the struct, the inner braces belong to the array field:

```
struct Palette
    byte color[4]
end

struct Track
    word note[3]
end

; CORRECT — outer {} = struct init, inner {} = array field init
Palette sky = {{ 10, 20, 30, 40 }}

; WRONG — the compiler rejects this (1 field but 4 values)
; Palette sky = { 10, 20, 30, 40 }   ; ERROR

; Access uses s.field[index] with constant or variable index
proc use_palette()
    Palette pal = {{ 1, 2, 3, 4 }}
    byte i = 2

    pal.color[0] = 100          ; constant-index write
    byte v = pal.color[0]       ; constant-index read

    pal.color[i] = 55           ; variable-index write
    byte u = pal.color[i]       ; variable-index read
```

```
    ; Loop over an array field
    byte sum = 0
    byte j
    for j = 0 to 4
        sum = sum + pal.color[j]
    end

    ; Struct copy also copies array fields
    Palette dst
    dst = pal                   ; copies all 4 bytes of color[]
end
```

Word array fields work the same way, with element offsets scaled automatically:

```
proc use_track()
    Track t = {{ 440, 880, 1760 }}
    word w = 0
    t.note[0] = 500
    byte k = 1
    t.note[k] = 1000
    w = t.note[k]               ; w == 1000
end
```

Struct Arrays

Arrays of structs are fully supported with initialization:

```
struct Enemy
    byte x
    byte y
    byte health
end

; Array of 10 enemies
Enemy enemies[10]

; Initialize array with values
Enemy level_enemies[3] = {
    { 10, 20, 100 },
    { 30, 40, 80 },
    { 50, 60, 120 }
}

proc update_enemies()
    byte i
    for i = 0 to 2
        enemies[i].x = enemies[i].x + 1
        enemies[i].health = enemies[i].health - 5
```

```
    end
end
```

Note: Whole struct-array copy (`dst = src`) is **not supported**. The compiler will reject it with an error. To copy a struct array, use a for loop:

```
Enemy src[3] = {{ 10, 20, 100 }, { 30, 40, 80 }, { 50, 60, 120 }}
Enemy dst[3]
byte i
for i = 0 to 3
    dst[i] = src[i]     ; copies one struct element at a time
end
```

Struct Address-Of Operator

Get the address of a struct or struct field using `@`:

```
proc struct_addressing()
    Point p = { 50, 100 }

    word p_addr = @p                ; Address of entire struct
    word x_addr = @p.x              ; Address of p.x field
    word y_addr = @p.y              ; Address of p.y field

    Enemy enemies[10]
    word enemy_addr = @enemies[0]   ; Address of first enemy
    word health_addr = @enemies[0].health
end
```

Const Structs

Mark structs as const to prevent modification:

```
const Point origin = { 0, 0 }
origin.x = 10               ; ERROR: Cannot modify field of const struct
```

All fields of a const struct are immutable at runtime:

```
struct Config
    byte mode
    byte flags
end

const Config default_config = { 1, 0 }
```

```
proc setup()
    byte mode = default_config.mode     ; OK - reading const field
    default_config.flags = 1            ; ERROR - modifying const struct
field
end
```

Struct Function Parameters

```
func byte distance(Point p1, Point p2)
    byte dx = p1.x - p2.x
    byte dy = p1.y - p2.y
    ; ... calculate distance
    return 0
end

proc use_structs()
    Point a = { 10, 20 }
    Point b = { 30, 40 }

    byte dist = distance(a, b)
end

Struct parameters are passed by value (copied). See the deep-copy rules in
[Parameter Passing](ZAP_LANGUAGE_REFERENCE.md#parameter-passing).

You can also pass a struct literal directly: `distance({ 10, 20 }, { 30, 40
})`.
```

Struct Function Return

Functions can return struct values:

```
func Point add_points(Point p1, Point p2)
    Point result = { p1.x + p2.x, p1.y + p2.y }
    return result
end

proc combine_points()
    Point a = { 5, 10 }
    Point b = { 15, 20 }
    Point c = add_points(a, b)     ; c = { 20, 30 }
end
```

Field access on function call result

A single field can be read directly from the return value of a struct-returning function, without first storing the whole struct:

```
func Point make_point(byte px, byte py)
    Point p
    p.x = px
    p.y = py
    return p
end

proc use_fields()
    ; Read a field directly from the call result
    byte bx = make_point(10, 20).x      ; bx = 10
    byte by = make_point(10, 20).y      ; by = 20

    ; Assign to a struct field from a call result field
    Point dest
    dest.x = make_point(3, 4).x        ; dest.x = 3
    dest.y = make_point(3, 4).y        ; dest.y = 4

    ; Works with word fields too
    ; (struct with word wx, word wy)
    word w = make_wvec(100, 200).wx    ; w = 100
end
```

The call is evaluated each time: `make_point(10, 20).x` and `make_point(10, 20).y` each emit a separate `JSR`. To read multiple fields from a single call, declare the struct variable with a call initializer at the top of the proc (declarations must precede statements):

```
proc use_fields()
    Point tmp = make_point(10, 20)  ; single call — declaration with call
initializer
    byte bx                         ; other declarations...

    ; now use tmp.x and tmp.y without calling again
    bx = tmp.x                      ; bx = 10
    ; ...
end
```

Note: ZAP declarations must appear before any executable statements in the proc body. `Point tmp = make_point(10, 20)` is valid only at the top of the proc, not after statements.

Pointers to Structs

Create pointers to struct types:

```
struct Data
    byte value
    byte flag
end
```

```
proc struct_pointers()
    Data d = { 42, 1 }
    Data ^ptr = @d              ; Pointer to struct

    ; Access through pointer
    byte val = ptr^.value       ; Read field through pointer
    ptr^.flag = 0               ; Write field through pointer
end
```

Port Structs (#PORT struct with field-level #RD / #WR)

A struct type can be declared with `#PORT` to model a memory-mapped hardware device where each field corresponds to a hardware register. Field-level `#RD` and `#WR` modifiers control read/write access per field, and a field with no modifier inherits the struct-level default.

```
struct VIA #PORT
    byte ORB            ; no modifier → inherits struct-level: both read
and write
    byte ORA    #RD     ; read-only field (write is a compile error)
    byte CTRL   #WR     ; write-only field (read is a compile error)
    byte STATUS #RD #WR ; explicit both read and write
end

VIA VIA1 @$9C00 #PORT   ; instance at fixed hardware address
```

Field modifier rules:

Field declaration	Read allowed	Write allowed
`byte F`	yes (inherits struct `#PORT` default — both)	yes
`byte F #RD`	yes	**no** — compile error
`byte F #WR`	**no** — compile error	yes
`byte F #RD #WR`	yes	yes

Accessing fields:

```
proc use_via()
    byte v = VIA1.ORA       ; ok — #RD field: read allowed
    VIA1.CTRL = $FF         ; ok — #WR field: write allowed
    VIA1.ORB  = $01         ; ok — inherited default: both allowed
    v = VIA1.ORB            ; ok — read also allowed
    ; VIA1.ORA = 1          ; ERROR: write to read-only port
    ; v = VIA1.CTRL         ; ERROR: read from write-only port
end
```

Struct-level `#PORT` with direction defaults:

A struct can also carry `#RD` or `#WR` at the struct level to set the default for all unqualified fields:

```
struct ReadOnlyChip #PORT #RD
    byte STATUS         ; inherits #RD → read-only
    byte DATA   #RD #WR ; overrides to both
end
```

Instance declaration and direction override:

When the struct type already carries `#PORT`, the instance inherits port semantics automatically — the `#PORT` modifier on the instance is optional. You can additionally place `#RD` or `#WR` on the instance to override the direction for all fields that have no explicit field-level modifier:

Instance declaration	Unqualified field direction
`MyPort P @addr`	both read and write (inherits struct default)
`MyPort P @addr #PORT`	both read and write (same — explicit is redundant)
`MyPort P @addr #PORT #RD`	read-only (unqualified fields become read-only)
`MyPort P @addr #PORT #WR`	write-only (unqualified fields become write-only)

Field-level modifiers (`#RD`, `#WR`, `#RD #WR`) always override the instance direction:

```
struct Mixed #PORT
    byte DATA           ; no modifier — takes direction from instance
    byte REG  #WR       ; explicit #WR — always writable regardless of
instance
end

Mixed P1 @$A000 #PORT #RD   ; P1 is read-only overall
; P1.DATA = 1              ; ERROR: DATA falls back to instance #RD — not
writable
; P1.REG  = 1              ; OK:   REG has explicit #WR — overrides
instance direction
```

Constraint summary:

- `#PORT` on the instance variable is optional when the struct type already declares `#PORT`; including it adds redundant validation checks (address required, no array, no pointer, no initializer).
- The instance variable must have an explicit `@address`.
- Port variables cannot have initializers.
- `#RD` and `#WR` on an instance variable require `#PORT` on the same declaration; using them without `#PORT` is a compile error.

Pointers

Pointer Declaration

```
byte ^ptr           ; Pointer to byte
word ^addr          ; Pointer to word
```

Taking Addresses

Use the `@` operator (address-of) to get the address of a variable:

```
byte x = 42
byte ^ptr = @x      ; ptr now points to x
```

Note: Use `@` for taking addresses. The `^` symbol is only used for pointer type declarations and postfix dereference (`ptr^`).

Dereferencing

```
byte x = 42
byte ^ptr = @x
byte y = ptr^       ; y now = 42
ptr^ = 99           ; x now = 99
```

Pointer Arithmetic

Pointers support strict C-style addition and subtraction. Offsets are automatically scaled by the pointed-to type:

```
proc pointer_arithmetic()
    byte arr[] = {10, 20, 30, 40, 50}
    byte ^ptr = @arr        ; ptr points to arr[0]

    ; BYTE pointers: +1 moves 1 byte
    ptr = ptr + 1           ; Now points to arr[1]
    byte value = ptr^       ; value = 20

    word addresses[] = {$1000, $2000, $3000}
    word ^wptr = @addresses

    ; WORD pointers: +1 moves 2 bytes
    wptr = wptr + 1         ; Skips to next WORD
    word addr2 = wptr^      ; addr2 = $2000

    ; Pointer difference computes the distance in elements
```

```
    word diff = wptr - @addresses ; diff = 1
end
```

Supported Math Operations:

- **`PTR + INT`** / **`INT + PTR`**: Moves the pointer forward by `INT` elements (scaled by `sizeof(type)`).
- **`PTR - INT`**: Moves the pointer backward by `INT` elements (scaled by `sizeof(type)`).
- **`PTR - PTR`**: Computes the difference between two pointers in elements. Both pointers must be of the same type. Returns a `WORD`.

Unsupported Math Operations:

- `PTR + PTR`, multiplying pointers, dividing pointers, and all bitwise operations (`&`, `|`, `^`, `<<`, `>>`) on pointers are explicitly bounded by the semantic checker and will cause a compilation error.

Parenthesized Pointer Dereference

You can dereference the result of pointer arithmetic directly using `(expr)^` syntax:

```
byte arr[5] = {10, 20, 30, 40, 50}
byte ^ptr = @arr

; Write to arr[2] via computed pointer
(ptr + 2)^ = 99         ; arr[2] is now 99

; Read from computed pointer
byte val = (ptr + 3)^   ; val = 40

; Stride scaling works for all pointer types
word warr[3] = {$1111, $2222, $3333}
word ^wptr = @warr
(wptr + 1)^ = $ABCD     ; warr[1] = $ABCD (advances 2 bytes)

long larr[3] = {$11111111, $22222222, $33333333}
long ^lptr = @larr
(lptr + 1)^ = $DEADBEEF ; larr[1] = $DEADBEEF (advances 4 bytes)

; Struct field access via computed pointer
struct Point
    byte x
    byte y
end
Point pts[3] = {{1,2}, {3,4}, {5,6}}
Point ^sptr = @pts
(sptr + 1)^.x = 99      ; pts[1].x = 99 (advances sizeof(Point) bytes)

; Compound assignment also works
(ptr + 1)^ += 5         ; arr[1] = arr[1] + 5
```

Rules:

- The `^` is required immediately after `(expr)` — `(expr) = val` without `^` is not valid.
- Only `.field` access is allowed after `(expr)^` (for struct pointers).
- Writing through a pointer derived from a `const` address is a compile error:

```
const byte data[3] = {1, 2, 3}
byte ^p = @data
(@data + 1)^ = 99     ; ERROR: Cannot write through pointer to const
'DATA'
```

- Port (`#PORT`) write-permission checks do not apply to `(expr)^` writes, since the base variable cannot be statically determined from a computed pointer expression. Use `ptr^` (simple identifier) for port access where `#WR` checking is needed.

Pointer Comparisons

Pointers can be compared using relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`).

- You can compare a pointer against another pointer (even of different types).
- You can compare a pointer against the literal constant `0` (null check).
- You can compare a pointer against any **`word`** value — because pointers are 16-bit, `word` is the natural type for storing and testing pointer values. This enables proper null pointer checks using named constants.
- Comparing a pointer against a `byte` variable or a string reference is a compile error.

```
byte ^ptr = @some_var

; Valid: pointer vs pointer
if ptr == @other_var
    ...
end

; Valid: pointer vs literal zero
if ptr == 0
    ...
end

; Valid: pointer vs WORD (e.g. NULL constant)
if ptr == $0000
    ...
end
```

NULL Pointer Idiom

ZAP does not have a built-in `NULL` keyword, but you can define one as a `const word`:

```
const word NULL = $0000
```

```
proc main()
    byte ^ptr       ; uninitialized (will be in BSS, zero-initialized)

    ; Set pointer to null
    ptr = NULL

    ; Test if pointer is null
    if ptr == NULL
        ; handle null case
    end

    ; Test if pointer is NOT null
    if ptr != NULL
        ; safe to dereference
        byte value = ptr^
    end
end
```

Key rules for NULL pointer usage:

- `ptr = NULL` — assigning a `word` value to a pointer is always allowed.
- `ptr == NULL` — comparing a pointer with a `word` value is allowed (pointer and `word` are both 16-bit).
- `NULL == ptr` — reversed comparison also works.
- `ptr == 0` — literal zero comparison is also valid for null checks.
- `byte` variables cannot be compared directly to pointers (only `word` or literal `0` is allowed).

Pointer to Pointers (Limited)

```
byte ^ptr = @some_var
; Further pointer-to-pointer not directly supported
```

Fixed-Address Pointers

```
byte ^DISPLAY_LIST @$D402       ; Hardware register
word ^COUNTER @$0600             ; Fixed address, no ZP
```

These cannot be dereferenced if not in zero-page.

Pointer Constraints

- Regular pointers must fit in zero-page (0x00-0xFF for address storage)
- Fixed-address pointers are always at their specified location
- Dereferencing is only safe for zero-page pointers

Module System

.module Directive

Mark a file as a module that can be included:

```
; lib_math.zaplib
.module "lib_math"

func byte abs_diff(byte a, byte b)
    if a > b
        return a - b
    end
    return b - a
end
```

Notes:

- The module name in the `.module` directive **must** be enclosed in double quotes (e.g., `.module "lib_math"`). The compiler enforces this and will raise an error for unquoted module names.
- Files declared as modules should not define `PROC MAIN()`. The `main()` entry point belongs in the top-level program that includes modules, not in library modules.
- The `.module` directive itself only accepts a module name (string). Use declaration modifiers (`#NOEXPORT`, `#EXPORT`, `#KEEP`) on individual declarations, `proc`, or `func` to control export and keep behavior; module directives are not (currently) interpreted for modifiers.

.include Directive

Module constructors

Modules may declare an optional special procedure named `Constructor()` that is invoked at program initialization. Key points:

- Declaration: `PROC Constructor()` at top level inside a `.module "name"` file.
- Constructors are forbidden in non-module files and a compile-time error is raised if found.
- Constructors behave as if annotated `#KEEP #NOEXPORT` so they are preserved and not exported.
- The compiler mangles constructor names to `__CONSTRUCTOR__<module_name>` to avoid label collisions.
- Calls to all module constructors are emitted after global/static initialization as `JSR __CONSTRUCTOR__<module>`, in dependency order (deepest include first).

Example:

```
.module "drivers"
PROC Constructor()
    ; driver init
END
```

Include another module's declarations and functions:

```
; program.zap
.include "lib_math.zaplib"

proc main()
    byte result = abs_diff(10, 3)   ; Use included function
end
```

Module Search

Includes are resolved relative to the including file's directory:

```
project/
  main.zap          (includes "lib/math.zaplib")
  lib/
    math.zaplib
```

Multiple Includes

```
.include "lib_math.zaplib"
.include "lib_graphics.zaplib"
.include "lib_sound.zaplib"
```

Circular Dependency Detection

The compiler detects and reports circular includes:

```
; lib_a.zaplib
.include "lib_b.zaplib"

; lib_b.zaplib
.include "lib_a.zaplib"    ; ERROR: Circular dependency
```

Module Organization

Best practices:

```
.zaplib files - Library modules with utilities, no main()
.zap files    - Applications with main() procedure
```

Directives

ZAP supports several directives for conditional compilation, module management, and diagnostics. Directives start with a dot (`.`).

Preprocessor Directives

Directive	Description
`.define SYMBOL`	Defines a symbol for use in conditional blocks.
`.undef SYMBOL`	Undefines a previously defined symbol.
`.ifdef SYMBOL`	Compiles the following block only if `SYMBOL` is defined.
`.ifndef SYMBOL`	Compiles the following block only if `SYMBOL` is NOT defined.
`.else`	Alternative branch for `.ifdef`/`.ifndef`.
`.endif`	Ends a conditional block.

Example:

```
.define DEBUG

.ifdef DEBUG
    .info "Debug mode enabled"
.else
    .define OPTIMIZED
.endif
```

Module and Inclusion Directives

Directive	Description
`.module "name"`	Declares the current file as a module. Enclosed code is scoped to the module.
`.include "path/to/file.zap"`	Includes another ZAP source file.
`.incbin "path/to/file.bin"`	Includes a binary file directly into the output.

Example:

```
.module "graphics"
.include "utils.zap"
.incbin "sprite_data.bin"
```

Diagnostic Directives

Directive	Description
`.error "message"`	Emits a compilation error with the specified message and stops compilation.
`.warning "message"`	Emits a compilation warning but processing continues.
`.info "message"`	Emits an informational message during compilation.

Example:

```
.ifdef DEMO_VERSION
    .warning "Compiling demo version - features restricted"
.endif
```

Advanced Topics

Inline Assembly

Embed raw 6502 assembly:

```
proc assembly_example()
    asm
        LDA #10         ; Load accumulator with 10
        STA $D400       ; Store to hardware register
    end
end
```

Memory Layout

Variables are allocated to memory in this order:

1. **Pointers** - Zero-page (must fit)
2. **Byte variables** - Zero-page, then BSS
3. **Word variables** - Zero-page, then BSS
4. **Arrays/Strings** - BSS (high memory)
5. **Temporary variables** - Zero-page (TMP0-TMP4)

```
byte ^ptr           ; 2 bytes in zero-page (address storage)
byte x              ; 1 byte in zero-page
byte y              ; 1 byte in zero-page
byte arr[256]       ; 256 bytes in BSS
```

Zero-Page Allocation

Zero-page is precious (only 256 bytes). Compiler optimizes:

- Unused local variables are removed
- Unused global variables are removed
- Only necessary temporary variables (TMP0-TMP4) are emitted

Const Folding and Optimization

Constant expressions are evaluated at compile-time:

```
const byte SIZE = 100
byte arr[SIZE + 50]         ; Array size is 150
const byte X = 2 + 3 * 4    ; X = 14 (computed at compile time)

proc main()
    byte y = SIZE * 2       ; y = 200 (compile-time)
end
```

Dead Code Elimination

Unreachable code is removed:

```
proc dce_example()
    return
    ; This code never executes - removed by optimizer
    byte unused = 42

    if 0              ; Condition always false
        byte never_used = 1
    end
end
```

Control Flow Quirks

Static Local Variables

Local variables are NOT re-initialized on each call:

```
proc counter()
    byte count = 0   ; NOT re-initialized each call!
    count = count + 1
    putc(count + 48) ; Prints: 1, 2, 3, 4, ...
end

proc main()
    counter()    ; Prints 1
    counter()    ; Prints 2
```

```
    counter()    ; Prints 3
end
```

Solution: Use initialization flags or global variables:

```
byte initialized = 0

proc counter_correct()
    byte count

    if !initialized
        count = 0
        initialized = 1
    end

    count = count + 1
end
```

Identifiers Shadow Global Names

Local variables completely shadow globals with the same name:

```
byte global_x = 10

proc test()
    byte global_x = 20      ; Shadows global
    global_x = global_x + 1 ; Uses local (= 21)
    ; No way to access global_x from here
end
```

Atari-Specific Features

Hardware Registers

```
byte GTIA_HPOS0 @$D000      ; Atari GTIA player 0 horizontal position
byte GTIA_SIZE0 @$D008      ; Player/missile size
word ANTIC_DLIST @$D402     ; Display list pointer

proc move_player()
    GTIA_HPOS0 = 100        ; Move player 0 to X=100
end
```

Atari Memory Map

```
$0000-$00FF   Zero-page and stack
$0100-$3FFF   RAM (often used for display list, screen memory)
$4000-$FFFF   Extended RAM, cartridge ROM
$D000-$D4FF   Hardware registers
```

Type Mixing in Expressions

```
proc type_mixing()
    byte b = 100
    word w = 1000

    byte result1 = b + 10       ; 8-bit arithmetic
    word result2 = w + 10       ; 16-bit arithmetic
    word mixed = b + w          ; Byte promoted to word
end
```

Function Return Type Coercion

```
func byte get_byte()
    return 50
end

func word get_word()
    return 1000
end

proc main()
    byte b = get_byte()
    word w = get_word()
end
```

Common Patterns

Counted Loop with Break

```
proc count_until_condition()
    byte i, found = 0

    for i = 0 to 255
        if some_condition(i)
            found = 1
            break
        end
    end
```

```
    if found
        ; Found at index i
    end
end
```

Initialization on First Call

```
byte first_call = 1

proc initialize_once()
    byte data

    if first_call
        data = 0
        first_call = 0
    end
end
```

Table Lookup

```
byte lookup_table[] = {10, 20, 30, 40, 50}

proc lookup(byte index)
    byte value = lookup_table[index]
    return value
end
```

Pointer-Based Iteration

```
proc iterate_with_pointer()
    byte data[] = {1, 2, 3, 4, 5}
    byte ^ptr = @data
    byte i

    for i = 0 to 4
        byte value = ptr^
        ptr = ptr + 1
    end
end
```

Bit Testing

```
proc test_bit()
    byte flags = $0F

    if flags & $01
        ; Bit 0 is set
    end
end
```

Compilation and Linking

Compile to Assembly

```
python compiler.py program.zap -o program.s
```

Assemble (with cc65 tools)

```
ca65 -I lib -t none --cpu 65c02 -g program.s -o program.o
```

Link for Atari

```
ld65 -C cfg/my_atari.cfg program.o lib/atari/exehdr.o -o program.com
```

Full Build Script

```
#!/bin/bash
# Compile
python3 compiler.py program.zap -o program.s

# Assemble
ca65 -I lib -t none --cpu 65c02 -g program.s -o program.o
ca65 -I lib -t none --cpu 65c02 -g lib/atari/exehdr.s -o exehdr.o

# Link
ld65 -C cfg/my_atari.cfg program.o exehdr.o -o program.com
```

Error Messages and Debugging

Common Errors

"Undefined variable"

```
proc main()
    x = 5   ; ERROR: x not declared
end
```

"Duplicate declaration"

```
byte x = 5
byte x = 10  ; ERROR: x already declared
```

"Type mismatch"

```
byte b = some_word_value    ; May lose data warning
```

"Procedure not found"

```
proc main()
    undefined_proc()    ; ERROR: undefined_proc doesn't exist
end
```

Debugging Tips

1. Check generated assembly (.s file) for actual output
2. Use labels and comments to trace execution
3. Test with simple cases before complex logic
4. Verify variable addresses don't collide with hardware
5. Use fixed-address variables cautiously

Performance Considerations

8-bit Constraints

- Only 256 bytes of zero-page memory
- Multiplication/division use runtime routines (not hardware)
- Pointer operations limited to zero-page locations
- Array access requires computed addressing

Code Size vs Speed Tradeoffs

The compiler automatically optimizes:

- Short loops → inline code

- Long loops → loop routines
- Short strings → inline init
- Long strings → ROM copy loops

Optimization Levels

```
# Default
python compiler.py program.zap

# With peephole optimizations
python compiler.py --peepholes program.zap

# 65C02 optimizations (default)
python compiler.py program.zap

# For older 6502 systems
python compiler.py -6502 program.zap
```

Appendix: Complete Example Program

```
; Fibonacci sequence generator
; Outputs Fibonacci numbers

.include "lib_graphics.zaplib"

const byte MAX_FIBO = 20

func byte fibonacci(byte n)
    if n <= 1
        return n
    end
    return fibonacci(n - 1) + fibonacci(n - 2)
end

proc print_number(byte num)
    byte tens = num / 10
    byte ones = num % 10

    putc(tens + 48)
    putc(ones + 48)
    putc(' ')
end

proc main()
    byte i

    for i = 0 to MAX_FIBO
        byte result = fibonacci(i)
        print_number(result)
```

```
    end
end
```

Revision History

Date	Version	Changes
Jan 2026	1.0	Initial comprehensive reference

Additional Resources

- ZAP Language Grammar (EBNF)
- Project State and Implementation Details
- Quick Reference Guide

For questions, issues, or suggestions regarding the ZAP! language, please visit the repository:
https://github.com/Dushino/ZAP-compiler

{% endraw %}